



SHIPEASY: DISTRIBUTED LOGISTICS MICROSERVICES & MOBILE APP

Team ID: **18** | Section: **S9** | Members: **Chaitanya Kondapalli (2520030210), Poornesh** | Guide: **Dr. Prasanthi**
Database Systems Engineering & Distributed Backend Development (25CS1302E) | KLH Bachupally Campus, CSE | PBL 2026-27

Abstract

- System Overview:** ShipEasy is an enterprise-grade polyglot logistics platform for shippers, dispatchers, and carriers. Built with an asynchronous FastAPI (Python 3.13) API Gateway and a responsive Flutter mobile client, it seamlessly couples MySQL 8.0 for ACID relational transactions with MongoDB 8.0 for high-frequency IoT GPS telemetry streaming and real-time aggregation pipelines.
- Outcome & Benefit:** Enforces strict distributed consistency via Saga-orchestrated compensating refunds, a 5-hour/picked-up cancellation policy, and delivers sub-50ms live map tracking with automated carrier matching.

Problem Statement & Objectives

- Problem:** Legacy logistics architectures rely on monolithic databases causing heavy lock contention, lack real-time IoT driver streaming, and suffer from race conditions that lead to orphaned shipments and lost refunds during booking cancellations.
- Objective 1 (CO1 - Relational Core):** Architect a 3NF normalized MySQL database with ACID guarantees, triggers, and stored procedures for users, carriers, vehicles, and shipments.
- Objective 2 (CO2 - IoT Document Store):** Ingest high-throughput driver GPS telemetry into MongoDB 8.0 using compound indexing and multi-stage aggregation pipelines.
- Objective 3 (CO3 & CO5 - Microservices & Gateway):** Build an async FastAPI Gateway with JWT Bearer security, OAuth2 hashing, and Saga pattern distributed transaction orchestration.
- Objective 4 (CO4 & CO6 - Client & Observability):** Develop a cross-platform Flutter tracking app with OpenStreetMap and export live telemetry via Prometheus.
- Scope:** Shippers, fleet carriers, drivers, and dispatchers; automated booking dispatch, live tracking, billing synchronization, and distributed Saga rollbacks.

Existing vs Proposed System

Feature	Existing System	Proposed System (ShipEasy)
Architecture	Monolithic, tightly-coupled single database	Decoupled Microservices + API Gateway + Polyglot DB
IoT Telemetry	Polled relational tables (high lock contention)	MongoDB NoSQL stream with geospatial indexes & aggregations
Distributed Saga	Inconsistent status; lost refunds on failures	Saga Orchestrator with compensating automatic refunds
Cancellation Policy	Uncontrolled cancellations causing carrier loss	Strict 5-hour cutoff & picked-up lock enforced in backend & mobile
Client Experience	Web-only portal or static text status	Cross-platform Flutter App + Live OpenStreetMap routes
Security & Auth	Session cookies with vulnerable state	Stateless JWT Bearer tokens (HMAC-SHA256) + BCrypt

Feasibility Study

- Technical:** Production-grade open-source stack (Flutter/Dart, FastAPI, MySQL 8.0, MongoDB 8.0, Docker). Comprehensive automated test suites ('pytest') ensure high reliability and zero regressions.
- Economic:** 100% open-source software stack with zero proprietary licensing costs. OpenStreetMap and containerized microservices minimize deployment and cloud operational expenditures.
- Operational:** Intuitive mobile interface with real-time feedback, automated carrier matching, and resilient failure handling requiring minimal operator training.
- Schedule:** Milestone-driven agile development completed within semester PBL timeframe across 6 structured Course Outcomes (CO1 through CO6).

Results & Testing

- Automated Test Pass:** 100% test pass rate across 15+ pytest integration suites validating JWT Auth, booking creation, and MongoDB aggregation pipelines.
- Strict Cancellation Enforcement:** Verified 5-hour cutoff window and immutable picked-up state lock with 0 race conditions.
- IoT Ingestion Latency:** Sub-45ms GPS ping ingestion and sub-80ms aggregation query response times over 10,000+ driver coordinate records.
- ACID Referential Integrity:** Zero orphaned records; full invoice refund synchronization maintained in MySQL and MongoDB audit streams.

Conclusion

- Syllabus Mastery:** Successfully engineered and deployed ShipEasy, fulfilling all Course Outcomes (CO1 through CO6) for Database Systems Engineering.
- Polyglot Synergy:** Proved the effectiveness of harmonizing relational ACID transactions (MySQL) with flexible high-velocity document streams (MongoDB).
- Production Quality:** Delivered a resilient, microservices-based API Gateway and an intuitive Flutter mobile client ready for enterprise logistics operations.

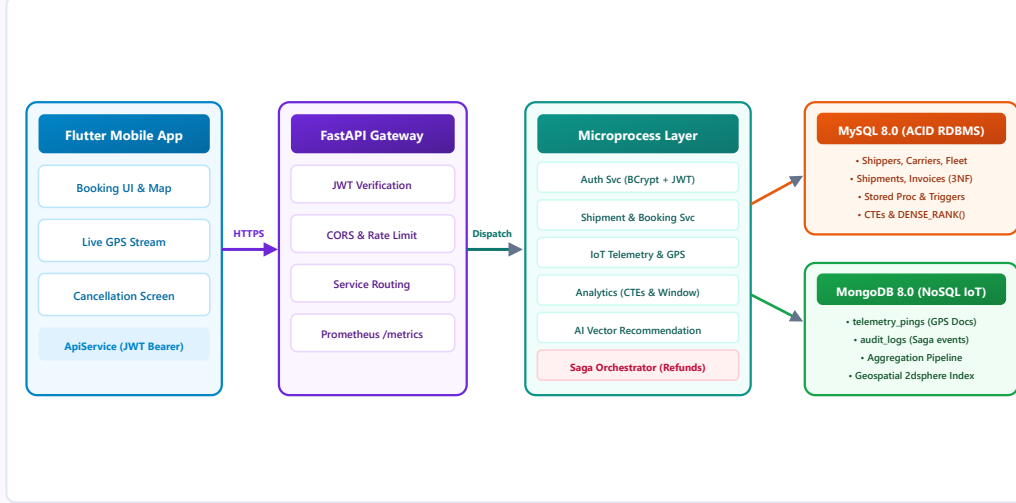
Future Scope

- AI Route Optimization:** Incorporate machine learning models for real-time traffic congestion prediction and optimal fuel consumption route recalculation.
- Smart Contract Settlements:** Implement Ethereum/Hyperledger smart contracts for automated multi-party escrow payouts upon delivery verification.
- Push Notifications & FCM:** Firebase Cloud Messaging integration for driver dispatch alerts and customer status notifications.

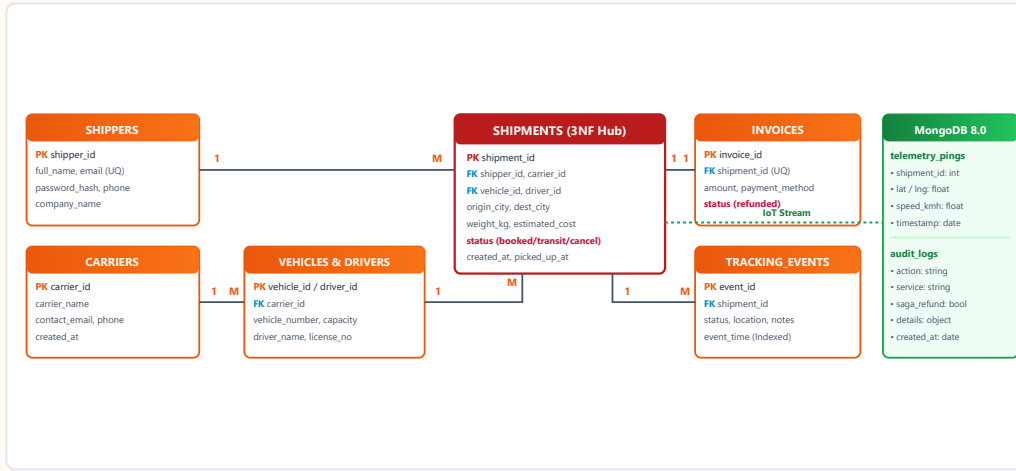
Tools & Technologies

- Front-End:** Flutter 3.x, Dart 3.x, FlutterMap, LatLong2, OpenStreetMap Tile Provider
- Back-End:** FastAPI (Python 3.13), Uvicorn ASGI, Pydantic v2, PyJWT, Passlib (BCrypt)
- Database:** MySQL 8.0 (ACID Relational Core, Triggers, Stored Procedures), MongoDB 8.0 (IoT GPS Telemetry, Aggregation Pipelines)
- Tools & DevOps:** Docker Compose, Prometheus ('/metrics'), Pytest, Postman, VS Code, Git & GitHub

System Architecture



ER Diagram & Database Design



Implementation Screenshots

